Code Summary: JavaScript

The scripts of the eight web tools in the Pi Zero 2 W build: how much script each page carries, which parts of the tool it talks to, how it keeps itself up to date, where it stores settings in the browser, and how it writes markup safely.

8 PAGES | ~1.5 MB OF SCRIPT | STAGE 3

SUMMARY · READ-ONLY REVIEW

Where the scripts are

The same eight pages as Stages 1 and 2. Script is kept two ways: literal `<script>` blocks in the page, and Python string constants (names containing JS, SCRIPT or THEME) that are joined when the page is served.

1.1 How the counts were made

- Both forms were collected for each tool. Script blocks that load an outside file were checked for; none was found.
- Function counts are approximate. They count `function` declarations and arrow functions bound to a name.
- Calls were counted in the text: `fetch(`, the page's own request helpers, `innerHTML` writes, timers and browser storage. Nothing was run in a browser.

Size and features per page

PAGE	PARTS	CHARACTERS	FUNCTIONS	FETCH CALLS	INNERHTML	NAMED ESCAPE HELPER
Node Controller	5	199,309	~275	35	47	yes (`esc`)
SysMon	3	260,808	~386	4 (plus helpers)	100	yes (`esc`)
instmon	15	151,712	~161	26	31	yes (`escHtml`)
wifimon	2	78,271	~119	2 (plus helpers)	0	not needed
44helper	36	406,136	~497	126	111	yes (per section)
Node Talker	3	208,899	~396	2 (plus helpers)	20	no
Node Recorder	3	54,975	~123	1 (plus helpers)	8	no
Pibrary	3	141,060	~221	19	2	no

"Plus helpers" means the page makes its requests through wrapper functions, so the direct `fetch` count is low. The endpoint list in Part 3 comes from the request paths named in the code.

2.1 Browser features

PAGE	LIVE CONNECTION	AUDIO AND MEDIA	BROWSER STORAGE	TIMERS
Node Controller	none	none	session storage (10 uses)	3 s, 10 s and 60 s
SysMon	none	none	session storage (7 uses)	1.5 s to 30 s
instmon	none	none	session storage (4 uses)	1 s to 15 s
wifimon	none	none	local storage, 4 keys	1 s and 5 s
44helper	WebSocket for the terminal	none	none	3 s and 5 s, plus elapsed-time timers
Node Talker	WebSocket for listen and talk	microphone, audio processing, media session	local storage, about 10 keys	250 ms to 30 s
Node Recorder	none	none	none	2 s and 30 s
Pibrary	none	media element	local storage (prefix `pibrary.`) and session storage	1 s to 300 s

Node Controller and SysMon

3.1 Node Controller

- **Size and shape:** five script parts. The main script is about 198,000 characters and holds roughly 275 functions. Three of the five parts are small placeholder scripts.
- **Requests:** actions go to `/api/action`, settings to `/api/config`, login to `/api/login`. Reference lists come from `/api/talkgroups`, `/api/xlx-reflectors`, `/api/m17-reflectors`, `/api/phone`, `/api/recording` and `/api/fcs-hosts`.
- **Keeping up to date:** the recording status is checked every 3 seconds, the analog bridge status every 10 seconds, and the keyed state is polled. The push-to-talk audio poll also runs every 3 seconds.
- **Inline handlers:** 167 `on...` attributes in the markup, so the page depends on `'unsafe-inline'` (see Part 5).

3.2 SysMon

- **Size:** the largest script of the eight, about 260,000 characters and about 386 functions.
- **Requests:** the page talks to about 50 paths. The most used are `/api/svc` (services), `/api/hardware`, `/api/config`, `/api/dvswitch/file`, `/api/nodelisten`, `/api/phone/routertest`, `/api/sdcard/test` and `/api/pinned`. The file editors use `/api/asterisk/file`, `/api/allmon3/file` and `/api/appconf/file`.
- **Keeping up to date:** refresh timers run from 1.5 seconds to 30 seconds.
- **Markup:** about 100 `innerHTML` writes, and 234 inline handlers. A shared `esc` function is used for values put into markup.

instmon, wifimon and 44helper

4.1 instmon

- **Size:** fifteen script parts, one for each feature: the log poll, status toasts, the config installer, the editor, actions, login, start-up, USB import, documents, and the Suite Log. The largest part is about 65,000 characters.
- **Requests:** actions go to `/api/action`. The disk-image feature has the largest group of paths (drives, format, backup, clone, job status, eject and cancel). Logs go to `/api/log` and `/api/suitelog/status`.
- **Jobs:** installs and disk-image jobs are followed by polling, at intervals from 1 to 15 seconds.

4.2 wifimon

- **Size:** two script parts, about 78,000 characters and about 119 functions. The page uses the most event listeners of the eight (70), and no `innerHTML` at all.
- **Requests:** status comes from `/api/status`. Settings go to `/api/watchdogs`, `/api/alerts` and `/api/country`. Network actions are under `/api/device/`, `/api/hotspot/`, `/api/change/` (keep and undo) and `/api/install/`.
- **Timers:** the pending-change and hotspot displays redraw every second. The install status is polled every 5 seconds.
- **Browser storage:** four keys with the `wifimon.` prefix. They are named for "seen" jobs, reports and results and for the details toggle. They appear to record what the browser has already shown, so a banner is not repeated. Confirm this in the code before the book states it.

4.3 44helper

- **Size:** the largest page of all: 36 script parts, about 406,000 characters and about 497 functions. Most tabs have their own script part, and the parts are joined in order when the page is built.
- **Requests:** the page makes about 126 direct fetch calls, and its paths follow a pattern: `/api/action_status` (the status of a running action), `/api/pi_install/action`, `/api/firewall/action`, `/api/router_install/action`, `/api/asl3/action`, `/api/dvswitch/action`, and many others for each tab.
- **Terminal:** the one WebSocket connects to `/api/term/ws` with a one-time ticket. This is the shell the Terminal button opens. The manual says it is for the local network and needs the root password (manual; see the Watchdogs, Guardrails, Locks and Controls summary).
- **Keeping up to date:** status checks at 3 and 5 seconds, with elapsed-time timers on running jobs.
- **Markup:** about 111 `innerHTML` writes, and 242 inline handlers. Each section has its own escape helper (for example `cfgEsc`, `htEsc`, `ndEscapeHtml` and `dvEscapeAttr`).

Node Talker, Node Recorder and Pibrary

5.1 Node Talker

- **Size:** about 208,000 characters and about 396 functions, with 125 event listeners.
- **Live link:** a WebSocket to `/api/talk` (`wss://` when the page is on HTTPS) is the live link the page uses for talk and listen.
- **Audio:** the page uses the microphone (`getUserMedia`, five references), audio processing (`AudioContext` and `AudioWorklet`), and the operating system's media session. Its audio code is the part to test on each browser and phone.
- **Requests:** connect and disconnect go to `/api/connect` and `/api/disconnect`, and the balance and restore actions to `/api/balance/`. Settings go to `/api/config`, which is called 15 times.
- **Browser storage:** about ten keys with the `nl` prefix hold the page's settings (for example push-to-talk, microphone, playback, headset, keep-awake and the automatic balance). They are kept per browser.
- **Markup:** 20 `innerHTML` writes, but no escape helper with a named function. Each write should be checked for values that come from outside.

5.2 Node Recorder

- **Size:** about 55,000 characters and about 123 functions, the smallest script of the eight.
- **Requests:** the schedule is saved to `/api/schedule/set`, and recordings are converted through `/api/convert` (with `/api/convert/install`). Reboot and shutdown are available from the page through `/api/system/reboot` and `/api/system/shutdown`, so the page can switch the node off.
- **Keeping up to date:** status is checked every 2 seconds, and a slower check runs every 30 seconds.
- **Markup:** 8 `innerHTML` writes, and no named escape helper.
- Node Recorder's script is close to Node Talker's, and the two have been edited separately (see the CSS summary). A fix to shared behaviour should be checked in both.

5.3 Pibrary

- **Size:** about 141,000 characters and about 221 functions, with 140 event listeners.
- **Requests:** the library is read through `/api/lib/list`, `/api/lib/file`, `/api/lib/art` and `/api/lib/recent`. Resume positions go to `/api/positions`. Imports use `/api/import/list` and `/api/import/copy`, and backups use `/api/config/backup` and `/api/config/restore`.
- **Browser storage:** local storage with the `pibrary.` prefix, plus session storage.
- **Timers:** from 1 second (the player) to 300 seconds (a slow check).
- **Markup:** two `innerHTML` writes, both built from strings. No named escape helper.

What the scripts have in common

6.1 Request paths

Requests are JSON calls to `/api/` paths, usually of the form `/api/<feature>/<action>`. Each tool has its own set. Some tools use a generic `/api/action` path for many actions, and others name each action as its own path.

6.2 Polling and job progress

Every page refreshes by timer rather than by push, except the two WebSockets. Long jobs (installs, disk-image work, the tunnel steps) are started with one request and then followed by a status request, polled at a set interval. The intervals in Part 2 show where the timers are, which is the place to look when tuning.

6.3 Login and session

Pages log in through `/api/login` (Node Controller, Node Talker, Node Recorder and others) and end the session through `/api/logout`. The page keeps no password. The session is held in a cookie.

Markup, inline script and the CSP

7.1 The Content-Security-Policy allows inline script

Five tools send a Content-Security-Policy: Node Controller, wifimon, instmon, 44helper and Pibrary. Each allows inline script, with `'unsafe-inline'` in `script-src`. The policy therefore does not block injected script, and escaping of values is the main defence against cross-site scripting. SysMon, Node Talker and Node Recorder send no policy in the code.

7.2 Inline handlers

PAGE	INLINE `ON...` ATTRIBUTES IN MARKUP
44helper	242
SysMon	234
Node Controller	167
instmon	16
Pibrary	2
wifimon, Node Talker, Node Recorder	0

7.3 Writing markup from data

The counts of `innerHTML` writes that build markup from strings are a rough guide, not a verdict. Writes built from values that come from the node, from a file, or from a network user need escaping. Node Controller, SysMon, instmon and 44helper escape with named helpers. Node Talker, Node Recorder and Pibrary have no named helper, and their writes should be checked one by one.

7.4 No code is eval'd

No page uses `eval` or `new Function`. The scripts are plain code that the browser reads once.

Things the scripts show

8.1 The scripts are large

Taken together, the eight pages carry about 1.5 million characters of script. Most of it is in a few pages: 44helper, SysMon and Node Talker. Changes to those pages are the costliest to review.

8.2 Escaping is not uniform

Four pages have a named escape helper, three write markup without one, and wifimon writes none. The book should describe the rule (every value from outside is escaped) and say which pages are checked against it.

8.3 A policy that allows inline script

The policy does not stop injected script, as described in Part 7. Describe it accurately in the book and do not present it as a protection against script injection.

8.4 Timers set the Pi's load

Pages poll at intervals as short as 250 milliseconds (Node Talker) and 1 second (several pages). On a Pi Zero 2 W with the audio pages open, these timers and the audio code share the processor. The cost has not been measured here.

8.5 Stored settings are per browser

Node Talker, Pibrary and wifimon keep some settings in the browser. A setting changed on one device does not appear on another. The book should say where a setting lives: on the node, or in the browser.